

Computer Organization & Assembly Language (CSC-205)

Lecture 2: Variables, Registers and Data Movement

Google Classroom code:
p5vf35j

For text books, slides, assignments, course updates
etc.

Importance of Assembly Language

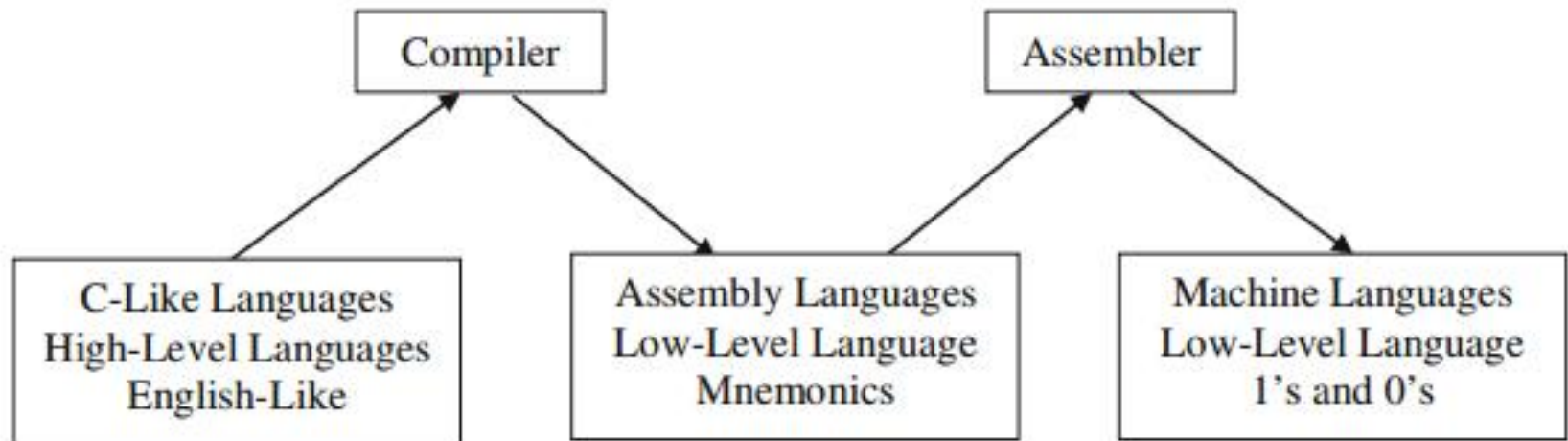
- Practically, the need for assembly language programming has decreased
- However, the need to understand it has not decreased
- Like programming in machine language, but easier – helps gain insight into computer organization and architecture
- Builds understanding of high level language implementations and compiler construction concepts

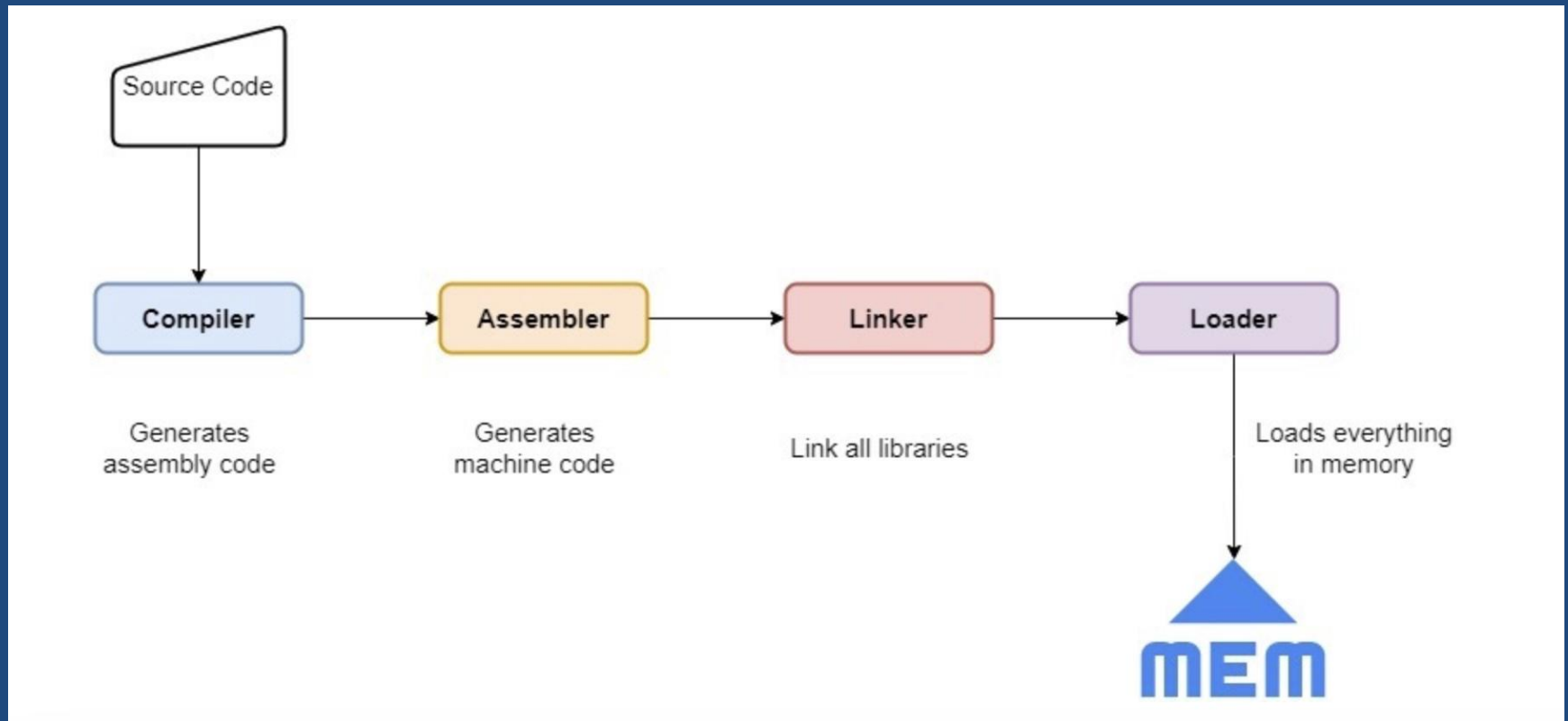
D/F b/w High level and Low level Lang?

- High-level languages are human-friendly. They are, thus, very easy to understand and learn by any programmer. C, FORTRAN, Java
- Closer to natural languages
- Make programs easier to read and write
- Low level languages, on the other hand are closer to machines
- Low-level languages are machine-friendly. They are, thus, very difficult to understand and learn by any human. Examples include assembly language, machine language

Assembly

- Midway, whereas programming in machine language is extremely tedious, assembly language is still manageable
- Assemblers convert assembly language to machine language





A compiler takes our source code and generates the corresponding assembly code. An assembler converts the assembly code to the machine code. A linker merges all the machine-code modules referenced in our code, whereas a loader moves the executable to RAM and lets it be executed by a CPU.

Are There Rules in Assembly Language?

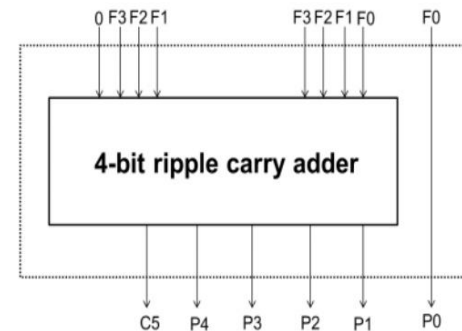
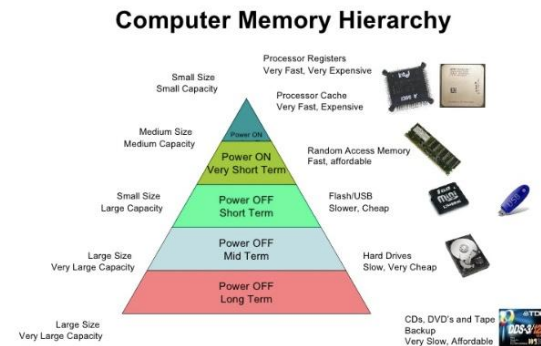
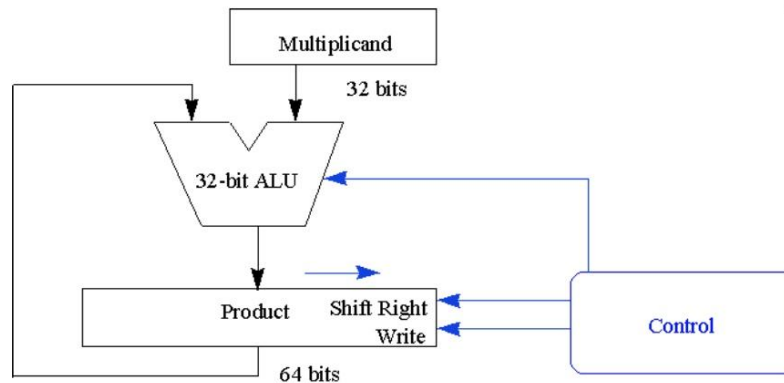
- Most rules in assembly language are based on physical limitations of the target processor and its machine language. The CPU, for example, requires two instruction operands to be the same size. Assembly language has fewer rules than C++ or Java because the latter use syntax rules to reduce unintended logic errors at the expense of low-level data access. Assembly language programmers can easily bypass restrictions characteristic of high-level languages.

Computer Organization

Organization is how features are implemented

Control signals, interfaces, memory technology.

e.g. Is there a hardware multiply unit or is it done by repeated addition?

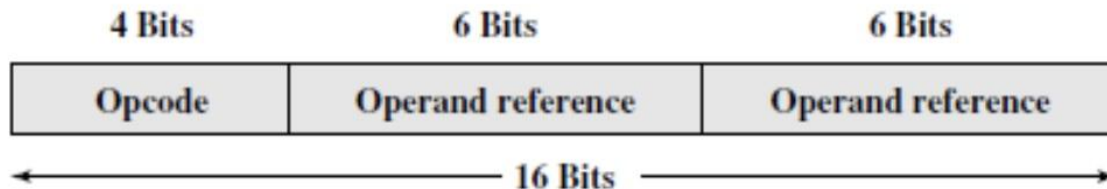


Computer Architecture

Architecture is those attributes visible to the programmer

Instruction set, number of bits used for data representation, I/O mechanisms, addressing techniques.

e.g. Is there a multiply instruction?

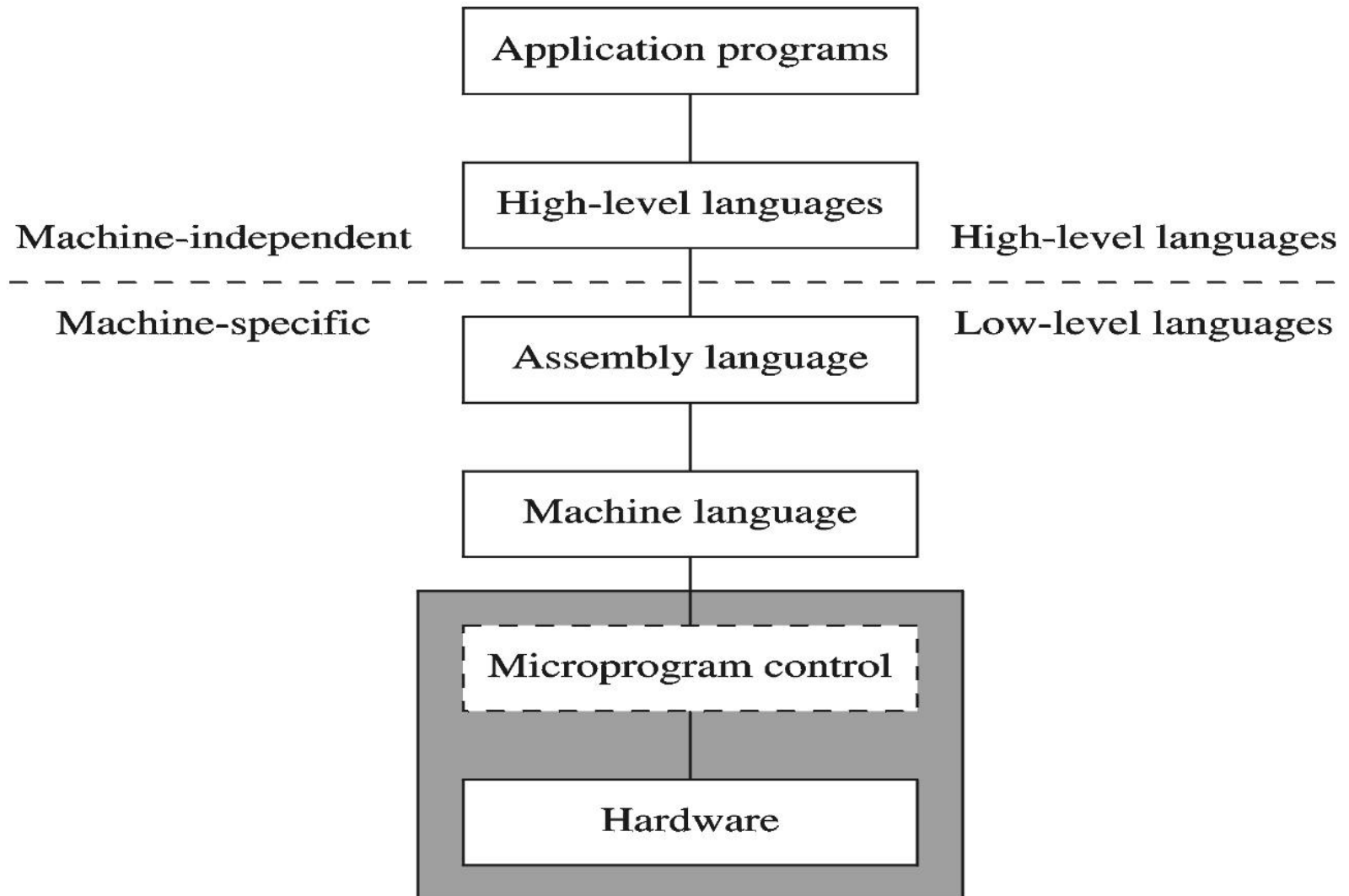


A Simple Instruction Format

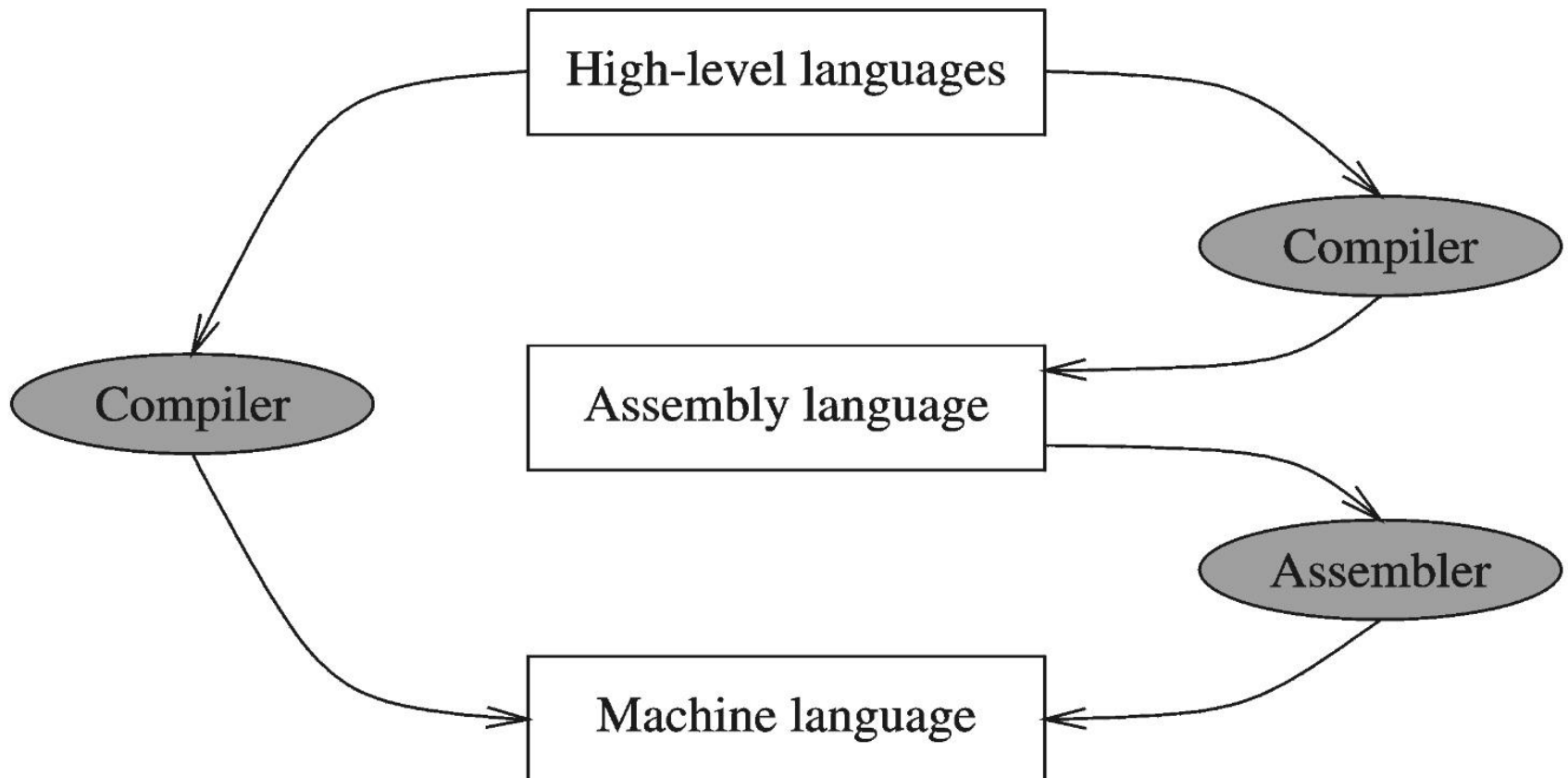
Is Assembly Language Portable?

- A language whose source programs can be compiled and run on a wide variety of computer systems is said to be portable.
- A C++ program, for example, will compile and run on just about any computer, unless it makes specific references to library functions that exist under a single operating system. A major feature of the Java language is that compiled programs run on nearly any computer system.
- Assembly language is not portable, because it is designed for a specific processor family. There are a number of different assembly languages widely used today, each based on a processor family.

A Hierarchy of Languages



Compiler and Assembler

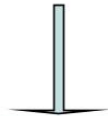


Translating Languages

English: D is assigned the sum of A times B plus 10.



High-Level Language: $D = A * B + 10$



A statement in a high-level language is translated typically into several machine-level instructions

Intel Assembly Language:

mov eax, A

mul B

add eax, 10

mov D, eax



Intel Machine Language:

A1 00404000

F7 25 00404004

83 C0 0A

A3 00404008

Advantages of High-Level Languages

- ❑ Program development is faster
 - ❑ High-level statements: fewer instructions to code
- ❑ Program maintenance is easier
 - ❑ For the same above reasons
- ❑ Programs are portable
 - ❑ Contain few machine-dependent details
 - Can be used with little or no modifications on different machines
 - ❑ Compiler translates to the target machine language
 - ❑ However, Assembly language programs are not portable

Why Learn Assembly Language?

- ❑ Two main reasons:
 - ❑ Accessibility to system hardware
 - ❑ Space and time efficiency
- ❑ Accessibility to system hardware
 - ❑ Assembly Language is useful for implementing system software
 - ❑ Also useful for small embedded system applications
- ❑ Space and Time efficiency
 - ❑ Understanding sources of program inefficiency
 - ❑ Tuning program performance
 - ❑ Writing compact code

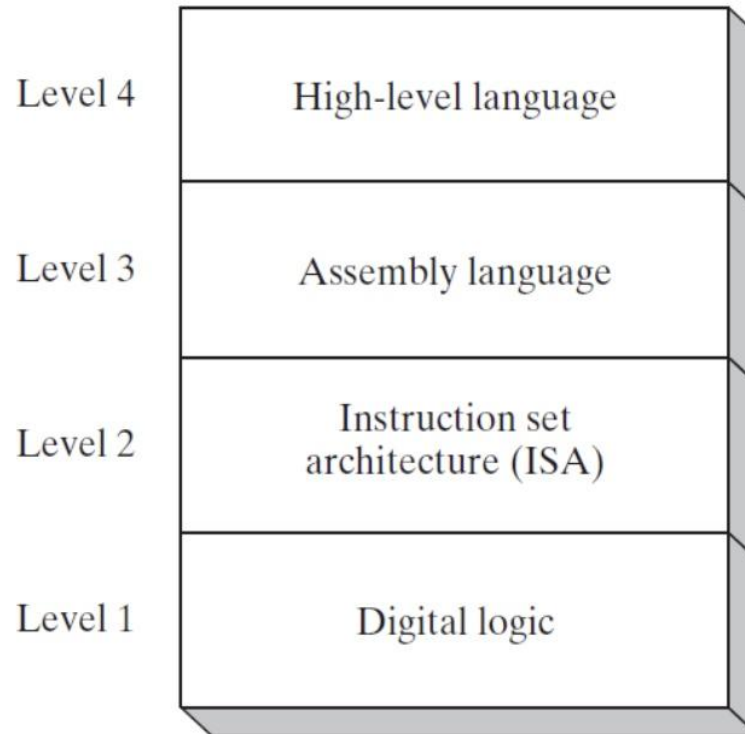
Assembly vs High-Level Languages

❖ Some representative types of applications:

Type of Application	High-Level Languages	Assembly Language
Business application software, written for single platform, medium to large size.	Formal structures make it easy to organize and maintain large sections of code.	Minimal formal structure, so one must be imposed by programmers who have varying levels of experience. This leads to difficulties maintaining existing code.
Hardware device driver.	Language may not provide for direct hardware access. Even if it does, awkward coding techniques must often be used, resulting in maintenance difficulties.	Hardware access is straightforward and simple. Easy to maintain when programs are short and well documented.
Business application written for multiple platforms (different operating systems).	Usually very portable. The source code can be recompiled on each target operating system with minimal changes.	Must be recoded separately for each platform, often using an assembler with a different syntax. Difficult to maintain.
Embedded systems and computer games requiring direct hardware access.	Produces too much executable code, and may not run efficiently.	Ideal, because the executable code is small and runs quickly.

Programmer's View of a Computer System

Increased level
of abstraction



Each level
hides the
details of the
level below it

Programmer's View – 2

❑ **Application Programs (Level 4)**

- ❑ Written in high-level programming languages
- ❑ Such as Java, C++, Pascal, Visual Basic . . .
- ❑ Programs compile into assembly language level (Level 4)

❑ **Assembly Language (Level 3)**

- ❑ Instruction mnemonics are used
- ❑ Have one-to-one correspondence to machine language
- ❑ Calls functions written at the operating system level
- ❑ Programs are translated into machine language

Programmer's View – 3

❑ **Instruction Set Architecture (Level 2)**

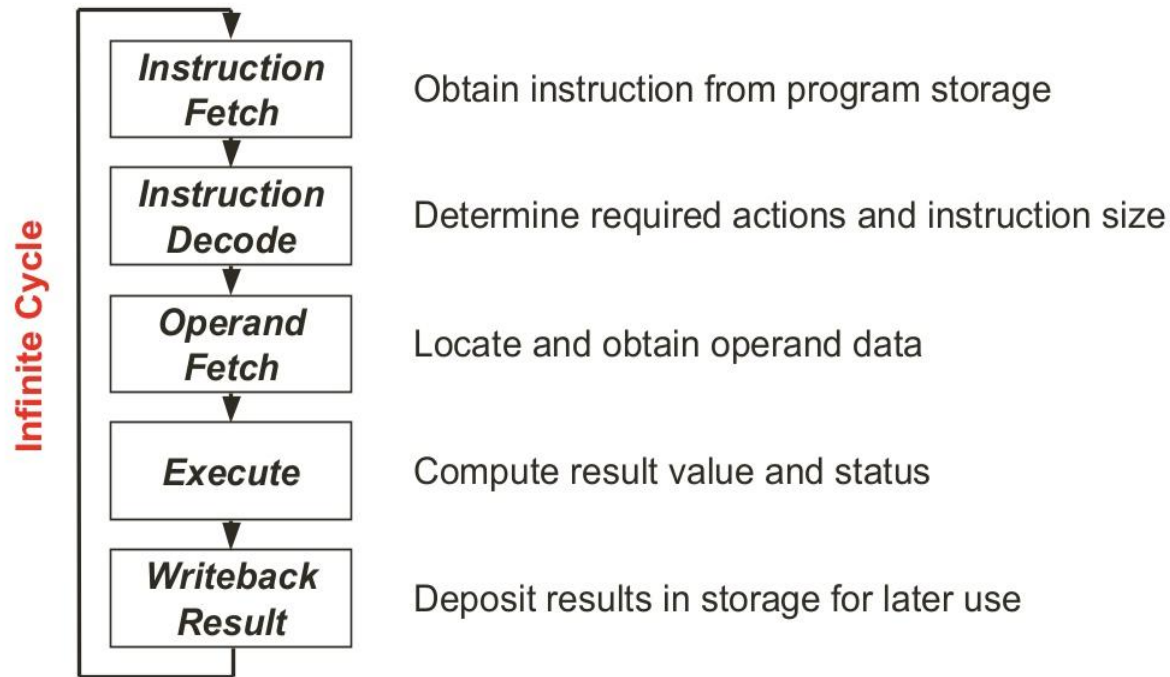
- ❑ Specifies how a processor functions
- ❑ Machine instructions, registers, and memory are exposed
- ❑ Machine language is executed by Level 1 (microarchitecture)

❑ **Digital Logic (Level 1)**

- ❑ Implements the microarchitecture
- ❑ Uses digital logic gates
- ❑ Logic gates are implemented using transistors

Instruction Execute Cycle

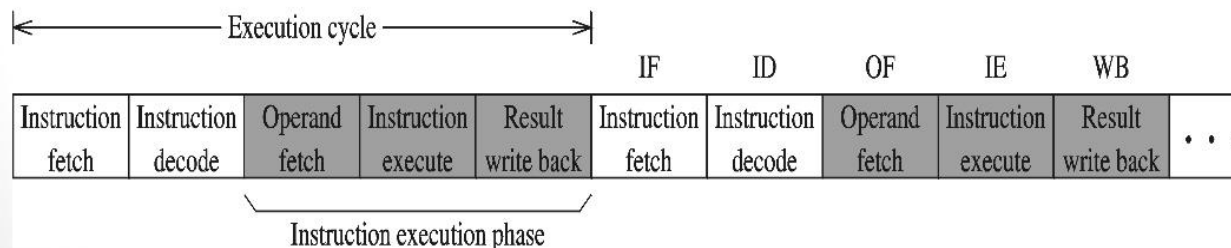
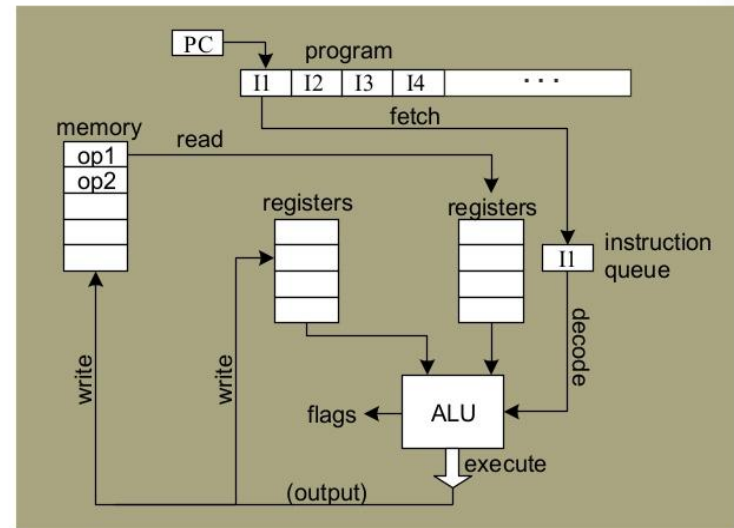
Instruction Execute Cycle



Instruction Execute Cycle.. cont'd

Instruction Execution Cycle – cont'd

- Instruction Fetch
- Instruction Decode
- Operand Fetch
- Execute
- Result Write back



Instruction Group

Instruction Group

- Data Movement Instructions

- These instructions are used to move data from one place to another.
- These places can be registers, memory, or even inside peripheral devices. Some examples are:

```
mov ax, bx
```

- Arithmetic and Logic Instructions

- Arithmetic instructions like addition, subtraction, multiplication, division and Logical instructions like logical and, logical or, logical xor, or complement are part of this group.
- Some examples are:

```
and ax, 1234
```

```
add bx, 0534
```

```
add bx, [1200]
```

Instruction Group

Instruction Group

- Program Control Instructions
 - These are instructions that control the program execution and flow by playing with the **instruction pointer (IP)** and altering its normal behavior to point to the next instruction.
 - Some examples are:
`cmp ax, 0`
`jne 1234`
- Special Instructions
 - Another group called special instructions works like the special service commandos. They allow changing specific processor behaviors and are used to play with it. They are used rarely but are certainly used in any meaningful program. Some examples are:
`cli`
`sti`
 - Where `cli` clears the **interrupt flag** and `sti` sets it

Reference

- ❑ Chapter #1
 - ❑ Assembly Language for x86 Processor 7th Edition
 - By Kip Irvine